SIMATS ENGINEERING
ASSESSMENT TOOL
(CO2 AT-1)

NAME : YUVANN NITHISH R
REGISTER NO:192411267
COURSE NAME: THEORY OF COMPUTATION
COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Code of Conduct:

I _Yuvann Nithish(192411267)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yuvann Nithish R

SIMATS ENGINEERING
ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet $\Sigma = \{0,1\}$ to validate passwords that **end with the pattern "01"**.Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications**.**

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a*b* can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L= \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

Question 1 — DFA for Passwords Ending With "01"

Design a Deterministic Finite Automaton (DFA) over Σ = {0, 1} that validates passwords ending with the pattern "01". Minimize the DFA and analyze how the optimized design improves efficiency in real hardware such as embedded security devices, ATM machines and authentication modules.

Step 1: Understanding the Requirement

A password string must be accepted only when its last two characters are "0" followed by "1". The DFA therefore needs to keep track of only the most recently seen character(s), because the accept/reject decision depends solely on the ending pattern, not on the entire history of the string.

Step 2: Formal Definition of the DFA

M = (Q, Σ, δ, q0, F), where:

- Q = {q0, q1, q2} — set of states
- Σ = {0, 1} — input alphabet
- q0 — start state (also represents "no useful suffix seen yet")
- F = {q2} — final/accepting state (last two symbols read were "0","1")
- δ — transition function, defined in the table below

Step 3: State Design Logic

- q0 : Start state / the last symbol read was NOT a 0 (i.e., string so far does not end in "0")
- q1 : The last symbol read WAS a 0 (a potential start of the pattern "01")
- q2 : The last two symbols read were exactly "0" then "1" — ACCEPT state

Step 4: Transition Table

Current State	Input 0	Input 1	State Type
q0 (Start)	q1	q0	Non-Final

Current State	Input 0	Input 1	State Type
q1	q1	q2	Non-Final
q2	q1	q0	Final (Accept)

Step 5: State Diagram (Flowchart)

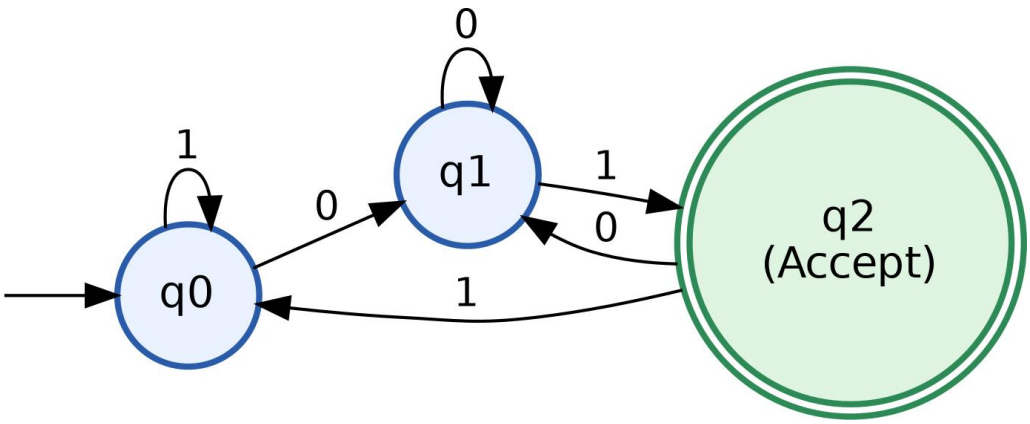

Figure 1.1 — Minimal DFA accepting all binary strings ending in "01"

Step 6: Tracing Sample Inputs

Input String	State Sequence	Result	Reason
1001	q0→q0→q1→q1→q2	Accepted	Ends in "01"
1010	q0→q0→q1→q2→q1	Rejected	Ends in "10"
0101	q0→q1→q2→q1→q2	Accepted	Ends in "01"
111	q0→q0→q0→q0	Rejected	Ends in "1", not "01"

Step 7: Minimization of the DFA (Equivalence / Partition Method)

Minimization is carried out using the Table-Filling (Myhill–Nerode partition refinement) method:

- Partition 0 (based on final vs non-final states): P0 = { {q2} , {q0, q1} }
- Check {q0, q1} for further split by observing transitions on each input symbol:

State	**On 0 → Group**	**On 1 → Group**
q0	q1 → Group B	q0 → Group B
q1	q1 → Group B	q2 → Group A

Since q0 and q1 move to different groups on input "1" (q0 stays in Group B, q1 moves to Group A), they are DISTINGUISHABLE. Hence the partition cannot be refined further:

- Final Partition = { {q0}, {q1}, {q2} } → 3 distinct equivalence classes

Conclusion: The DFA designed in Step 4 already has the minimum possible number of states (3). No further reduction is possible — it is the minimal DFA for this language.

Step 8: Analysis — Real-World Hardware Efficiency

- Fewer States = Fewer Flip-Flops: Each DFA state requires log2(|Q|) bits of memory in hardware. A 3-state minimal DFA needs only 2 flip-flops instead of more for a redundant design, directly reducing chip/silicon area.
- Lower Power Consumption: Fewer sequential logic elements switching on every clock cycle means lower dynamic power draw — critical for battery-powered embedded security tokens and POS/ATM terminals.
- Faster Response Time: A minimal state machine has a shorter critical path in its combinational transition logic, allowing a higher maximum clock frequency, so passwords/PINs are validated with lower latency.
- Lower Cost & Higher Reliability: Reduced gate count lowers manufacturing cost and reduces the probability of hardware faults, which is essential in authentication modules

Question 2 — Regular Expression & FA for Input "aaaabb"

Design a regular expression to validate the specific input pattern "aaaabb" over Σ = {a, b}, and analyze how regular languages/expressions validate user input in real web applications.

Step 1: Identify the Language

The requirement is to accept one exact fixed string only: L = { aaaabb }. Since there is no repetition, choice or optional part, the regular expression is simply the concatenation of the literal symbols in order.

Step 2: Regular Expression

Regular Expression: R = a . a . a . a . b . b which is written simply as:

R = aaaabb

Step 3: Formal Definition of the FA

M = (Q, Σ, δ, q0, F) where Q = {q0, q1, q2, q3, q4, q5, q6, qd}, Σ = {a, b}, start state = q0, F = {q6}, and qd is a dead/trap state used to reject any string that deviates from the exact pattern.

Step 4: Transition Table

State	On 'a'	On 'b'	Type
q0 (Start)	q1	qd	Non-Final
q1	q2	qd	Non-Final
q2	q3	qd	Non-Final
q3	q4	qd	Non-Final
q4	qd	q5	Non-Final
q5	qd	q6	Non-Final
q6	qd	qd	Final (Accept)

State	On 'a'	On 'b'	Type
qd (Dead)	qd	qd	Trap

Step 5: State Diagram (Flowchart)

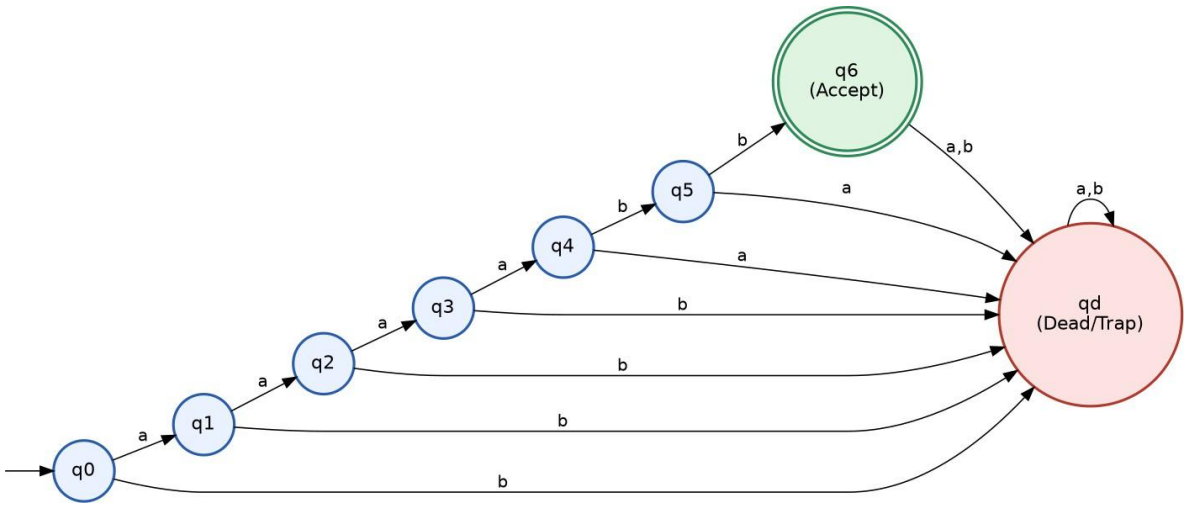

Figure 2.1 — Linear FA that accepts only the exact string "aaaabb"

Step 6: Step-by-Step Trace for Input "aaaabb"

Step	Current State	Symbol Read	Next State
1	q0	a	q1
2	q1	a	q2
3	q2	a	q3
4	q3	a	q4
5	q4	b	q5

Step	Current State	Symbol Read	Next State
6	q5	b	q6 (Accept)

Since the input is fully consumed and the machine halts in q6 ∈ F, the string "aaaabb" is ACCEPTED. Any deviation (extra/missing/wrong symbol) sends the machine to the dead state qd, which can never reach q6, so it is correctly REJECTED.

Step 7: Analysis — Regular Expressions in Real-World Web Validation

- Client-Side Form Validation: Browsers compile regex patterns (e.g., HTML5 "pattern" attribute, JavaScript RegExp) internally into finite automata that scan input in a single linear O(n) pass, giving instant feedback without server round-trips.
- Security & Sanitization: Regex-based FAs are used to whitelist only expected formats (emails, phone numbers, tokens), which blocks malformed or malicious input such as SQL-injection or script-injection payloads before they ever reach the backend.
- Performance: Because a DFA processes each character exactly once with O(1) work per character, regex validation scales efficiently even for high-traffic web applications handling thousands of form submissions per second.
- Consistency: Since regular expressions and finite automata are mathematically equivalent (Kleene's theorem), a pattern designed on paper (like R = aaaabb) translates directly and predictably into deterministic runtime behaviour in any regex engine.

Justification: Regular languages give web developers a precise, mathematically provable and computationally cheap way to describe "acceptable" input formats; the underlying FA guarantees that validation is exhaustive (every case is covered), deterministic (no ambiguity) and fast (linear time), which is exactly what is needed for real-time input validation.

Question 3 — Converting Regular Expression a*b* to an Equivalent FA

Demonstrate how the regular expression a*b* can be converted into an equivalent finite automaton, and explain why regular expressions and finite automata are equivalent models used in real-time pattern/intrusion detection systems.

Step 1: Break Down the Regular Expression

a*b* is the concatenation of two independent Kleene-star sub-expressions: R1 = a* (zero or more a's) and R2 = b* (zero or more b's), joined as R = R1 . R2. Thompson's Construction builds a small NFA for each part and then combines them.

Step 2: NFA for a* (Thompson's Star Rule)

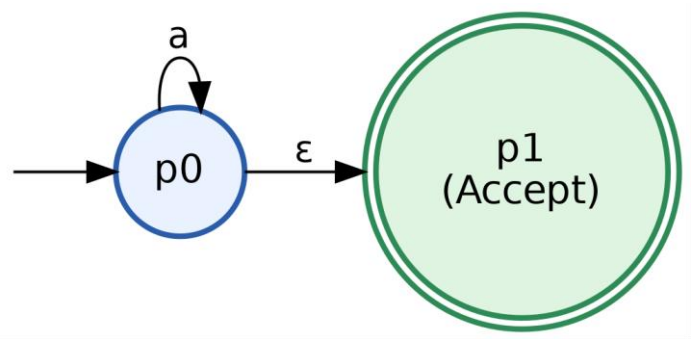

Figure 3.1 — NFA fragment for a : p0 is both start and accept, with a self-loop on 'a'*

Step 3: NFA for b* (Thompson's Star Rule)

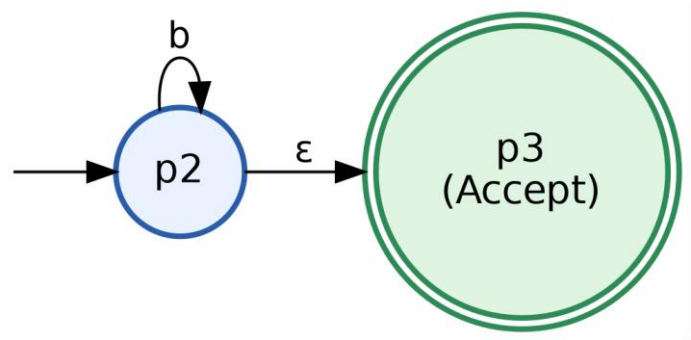

Figure 3.2 — NFA fragment for b : p2 is both start and accept, with a self-loop on 'b'*

Step 4: Concatenation — Join a* and b* with an ε-transition

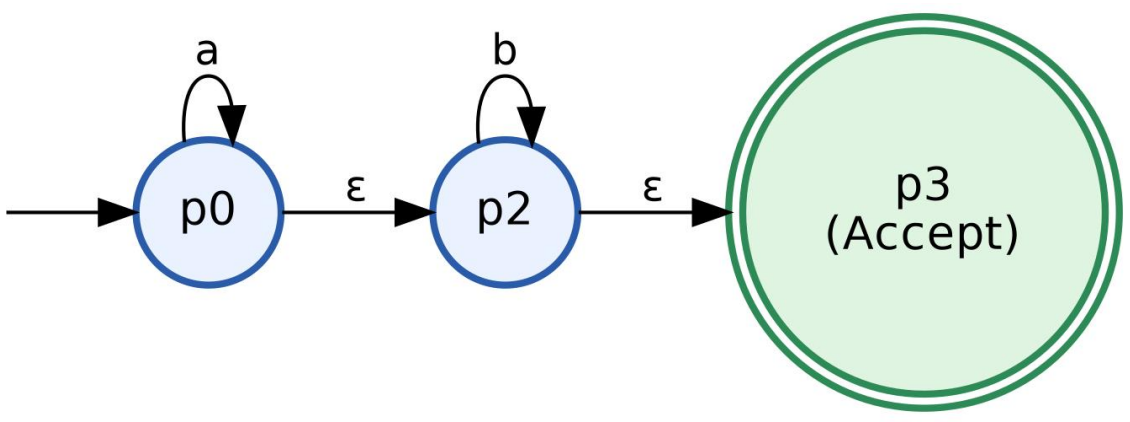

*Figure 3.3 — Combined ε-NFA for a*b* (Thompson's Concatenation Rule)*

Step 5: Subset Construction (ε-NFA → DFA)

Applying the subset construction algorithm, we compute ε-closures of state sets to form DFA states:

DFA State	ε-closure Set	On 'a'	On 'b'	Type
A	{p0, p2, p3}	→ A	→ B	Start, Accept
B	{p3}	→ C (dead)	→ B	Accept
C	{ } (dead)	→ C	→ C	Trap / Non-Final

Step 6: Final Minimized DFA for a*b*

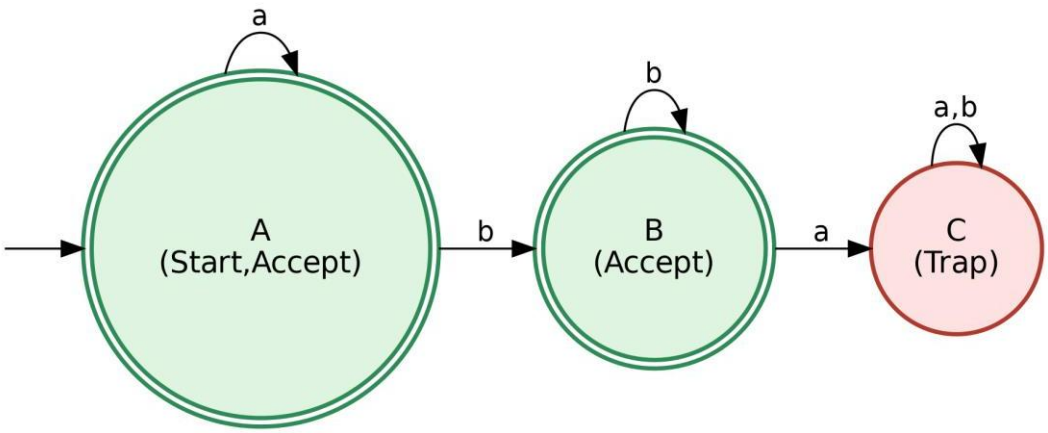

*Figure 3.4 — Equivalent minimal DFA recognizing L(a*b*) = { $a^i b^j$ | i, j ≥ 0 }*

Step 7: Why Regular Expressions and Finite Automata Are Equivalent

By Kleene's Theorem, every regular expression has an equivalent NFA (built inductively using Thompson's rules for union, concatenation and star), every NFA has an equivalent DFA (via subset construction), and every DFA can be converted back into a regular expression (via state elimination / Arden's method). This three-way equivalence — Regular Expression ⇔ NFA ⇔ DFA — means all three formalisms describe exactly the class of Regular Languages.

Step 8: Analysis — Use in Real-Time Detection Systems

- Intrusion Detection Systems (IDS): Signature patterns for known attacks are written as regular expressions, then compiled into DFAs (or combined multi-pattern automata like Aho-Corasick) so that network packets can be scanned in real time with a single linear pass.
- Compilers: The lexical analyzer (scanner) phase uses exactly this regex-to-DFA pipeline to tokenize source code into identifiers, keywords, and operators efficiently.
- Deterministic, Bounded-Time Behaviour: Because a DFA has exactly one transition per symbol, worst-case time is always O(n) and memory is fixed — a critical guarantee for real-time systems where unpredictable delays are unacceptable.

Justification: Building the FA in stages (NFA → ε-NFA → DFA) shows constructively why any regex can always be mechanically converted to hardware/software-friendly automata, giving real-time systems the speed and predictability of finite automata while retaining the human-readable convenience of writing detection rules as regular expressions.

Question 4 — Pumping Lemma: Proving L = {$a^n b^n$ | n ≥ 0} is Not Regular

Using the Pumping Lemma, demonstrate why L = {$a^n b^n$ | n ≥ 0} cannot be recognized by any finite automaton, and analyze the implications for modern compiler design.

Step 1: Statement of the Pumping Lemma for Regular Languages

If L is a regular language, then there exists an integer p ≥ 1 (the pumping length) such that every string w ∈ L with |w| ≥ p can be split into three parts w = xyz satisfying ALL of the following conditions:

- 1) |xy| ≤ p
- 2) |y| ≥ 1 (y is non-empty)
- 3) For every i ≥ 0, the string x y^i z must also belong to L

Step 2: Proof by Contradiction — Setup

Assume, for the sake of contradiction, that L = {$a^n b^n$ | n ≥ 0} IS regular. Then the Pumping Lemma must apply to it, and some pumping length p must exist.

Step 3: Choose a String to Pump

Choose w = $a^p b^p$. This string is in L (it has an equal number of a's and b's) and |w| = 2p ≥ p, so the Pumping Lemma conditions apply to w.

Step 4: Apply the Split Condition

Since w = xyz with |xy| ≤ p, and the first p symbols of w = $a^p b^p$ are ALL 'a's, both x and y must consist entirely of 'a' symbols only (no 'b' can appear inside x or y).

Step 5: Pump the String (i = 2)

Consider i = 2: the pumped string becomes x y^2 z. Since y contains only a's and |y| ≥ 1, this pumped string now has MORE a's than the original p, while the number of b's remains exactly p.

Step 6: Reach the Contradiction

Therefore x y^2 z = a^(p+|y|) b^p, which has strictly more a's than b's. This string is clearly NOT of the form $a^n b^n$, so x y^2 z ∉ L. But the Pumping Lemma requires x y^i z ∈ L for every i ≥ 0 — this is a direct contradiction.

Step 7: Conclusion

Since assuming L is regular leads to a contradiction, our assumption must be false. Therefore, $L = \{a^n b^n \mid n \geq 0\}$ is NOT a regular language — no finite automaton (DFA/NFA) can recognize it, because a finite automaton has only a fixed, finite number of states and cannot "count" and remember an arbitrarily large, unbounded number of a's in order to match it exactly against the number of b's.

Step 8: Proof Flowchart

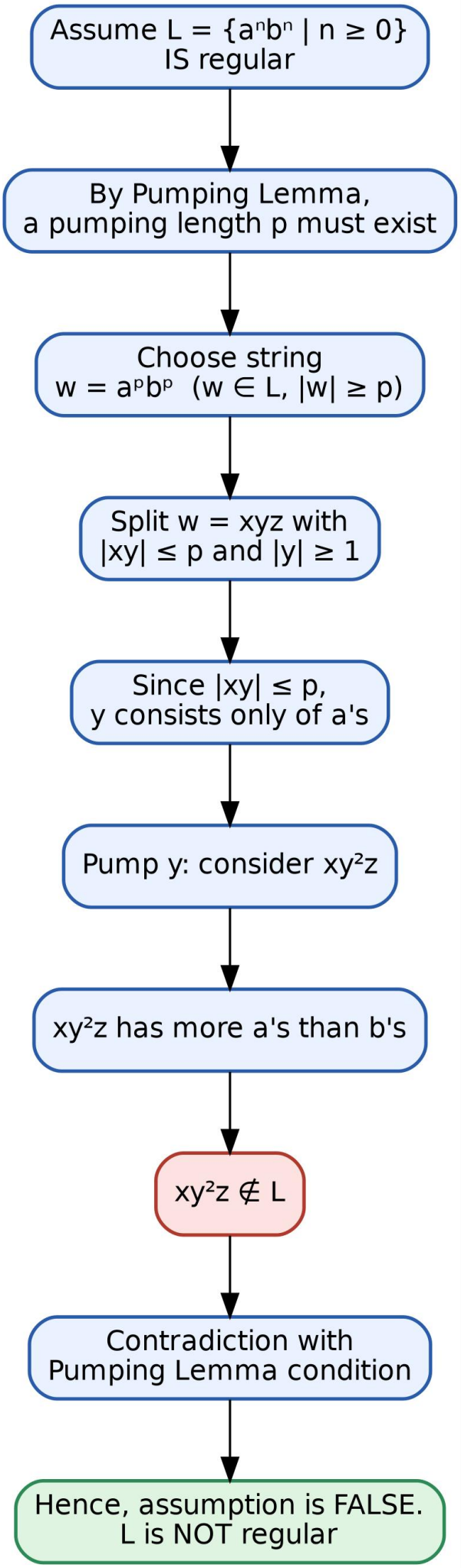
Assume L = {aⁿbⁿ | n ≥ 0}
IS regular
By Pumping Lemma,
a pumping length p must exist
Choose string
w = aᵖbᵖ (w ∈ L, |w| ≥ p)
Split w = xyz with
|xy| ≤ p and |y| ≥ 1
Since |xy| ≤ p,
y consists only of a's
Pump y: consider xy²z
xy²z has more a's than b's
xy²z ∉ L
Contradiction with
Pumping Lemma condition
Hence, assumption is FALSE.
L is NOT regular

Figure 4.1 — Flow of the Pumping Lemma contradiction proof

Step 9: Worked Numerical Example (p = 3)

Item	Value
Pumping length chosen	p = 3
String w = $a^p b^p$	aaabbb
Valid split (example)	x = ε, y = "a", z = "aabbb"
Pumped string (i = 2): xy^2z	aaaabbb
Is $xy^2z \in L$?	No — 4 a's but only 3 b's → contradiction

Step 10: Analysis — Implications for Modern Compiler Design

- Limits of the Lexer: Since finite automata (used in the lexical-analysis phase) cannot recognize languages requiring unbounded counting/matching, they cannot by themselves verify constructs like balanced parentheses ( ), brackets [ ], or braces { } — which are structurally the same problem as $a^n b^n$.
- Need for a Separate Parsing Phase: Compilers therefore use Context-Free Grammars and Pushdown Automata (which have an auxiliary stack for unbounded memory) in the syntax-analysis (parsing) phase to correctly validate nested and balanced structures.
- Two-Phase Compiler Architecture: This limitation is exactly why compiler design is split into a fast regex/DFA-based lexer (for tokens) followed by a more powerful CFG/PDA-based parser (for grammar/structure) — using the right computational model for the right sub-problem.

Justification: The Pumping Lemma formally proves the theoretical boundary of what finite automata can express. Recognizing this boundary is precisely what guides compiler designers to layer a stack-based parser on top of a finite-state lexer, ensuring correctness for recursive, nested language constructs that appear in every real programming language.

Question 5 — Closure Properties of Regular Languages

Explain how the closure properties of regular languages enable combining multiple pattern-matching/detection rules, and justify why the resulting language remains regular, ensuring efficient implementation using finite automata.

Step 1: What "Closure Property" Means

A class of languages is said to be "closed" under an operation if applying that operation to members of the class always produces another member of the same class. Regular Languages are closed under several important operations, each of which corresponds to a concrete automaton-construction technique.

Step 2: Key Closure Properties of Regular Languages

Operation	**Definition**	**Automaton Construction Technique**
Union (L1 ∪ L2)	Strings in L1 OR L2	New start state with ε-transitions to both FA1 and FA2 (Thompson's Union Rule)
Concatenation (L1 . L2)	Strings formed as w1w2, w1∈L1, w2∈L2	ε-transition from every accept state of FA1 to the start state of FA2
Kleene Star (L*)	Zero or more repetitions of strings from L	ε-loop back from accept states to the start state, plus a new accepting start state
Intersection (L1 ∩ L2)	Strings in BOTH L1 AND L2	Product Construction — states are pairs (state of FA1, state of FA2)
Complement ($\bar{L}$)	Strings NOT in L	Take a complete DFA for L and swap final / non-final states

Operation	Definition	Automaton Construction Technique
Reversal (L^R)	Strings of L written backwards	Reverse all edges; swap start and accept states

Step 3: Combining Multiple Detection Rules — Worked Example

Suppose Rule A (FA1) flags input containing a malicious keyword pattern, and Rule B (FA2) flags input exceeding an unsafe length pattern. Using the PRODUCT CONSTRUCTION for intersection (or union, depending on the required logic), a single combined automaton FA_AB can be built where each state is a pair (state-in-FA1, state-in-FA2):

- If the required logic is "flag if EITHER rule matches" → apply Union → mark a product state accepting if either component is accepting.
- If the required logic is "flag only if BOTH rules match" → apply Intersection → mark a product state accepting only if both components are accepting.

The number of states in the product automaton is at most |Q1| × |Q2|, which is still finite — this finiteness is the core reason the resulting automaton (and therefore the resulting language) remains regular.

Step 4: Closure-Combination Flowchart

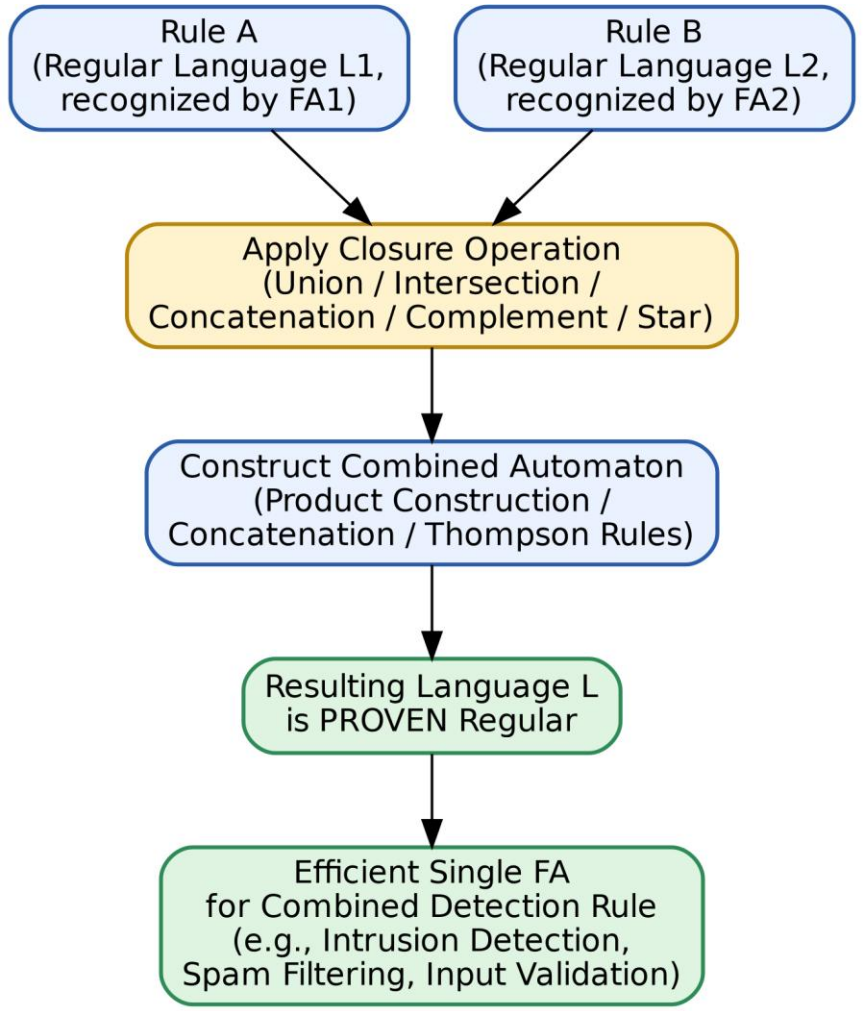

Figure 5.1 — Combining two rule-automata into one regular, single-pass detection automaton

Step 5: Why the Resulting Language Is Guaranteed to Remain Regular

- Finiteness is preserved: Every closure construction (union, intersection, concatenation, star, complement) produces an automaton with a FINITE number of states, built purely from the finite state sets of the input automata — never an unbounded/infinite structure.

- Determinism can always be restored: Even if a construction (like union via ε-NFA) temporarily produces a non-deterministic machine, subset construction always converts it back into an equivalent DFA, which is by definition regular.

- Algebraic closure: Since regular expressions themselves are closed under union (+), concatenation (.), and star (*), and every regular expression corresponds to some regular

language, any expression built by combining smaller regular expressions is itself regular by construction.

Step 6: Analysis — Real-World Application

- Intrusion Detection Systems: Multiple attack-signature automata are merged (via union) into one combined automaton so that network traffic is scanned only ONCE per packet instead of once per rule — dramatically improving throughput.

- Spam Filtering: Multiple keyword/pattern rules for spam detection are combined using union/intersection into a single filtering automaton, enabling real-time email scanning at scale.

- Input Validation: Complex validation logic (e.g., "must look like an email AND must not contain banned words") is implemented cleanly as an intersection of two simpler regular languages, each independently easy to design and test.

Justification: Because closure properties guarantee that combining any finite set of regular rules through union, intersection, concatenation, complement or star always yields another finite automaton, system designers can confidently build complex, composite detection/validation logic by combining simple, independently verified regular rules — while still guaranteeing efficient, single-pass, linear-time (O(n)) execution using one finite automaton, instead of needing to design one large, error-prone automaton from scratch or run multiple separate slower passes.